软 件 工 程 实 验 · 实 验 二

软件设计与建模

面向对象设计 · UML 建模 · UI 界面设计

校园食刻 —— 校园及周边外卖点餐与配送系统

课程名称	**软件工程实验**
项目名称	**校园食刻 • 外卖点餐与配送系统**
姓　　名	**周文朋**
学　　号	**241220167**
需求来源	**实验一需求规格说明（REQ-001 ~ REQ-025）**
仓库地址	**https://github.com/yui060814zwp-creator/SE-lab**
版本标签	**lab2-draft → lab2-final**
完成日期	**2026 年 11 月 12 日**

交付物：实验报告.pdf · UML/（11 张图 + PlantUML 源码）· UI界面/（16 屏）

目　录

一、项目与主要功能 3
1.1 项目与本次任务 3
1.2 软件的主要功能（表 1） 3

二、软件架构设计 4
2.1 架构风格与理由 4
2.2 分层与依赖规则（图 1） 4
2.3 模块划分（表 2） 5
2.4 需求覆盖（表 3） 5
2.5 模块之间的交互（表 4） 6
2.6 关键设计决策（表 5） 7

三、UML 建模 8
3.1 三类图与本项目的选型（表 6） 8
3.2 类图（图 2、图 3） 9
3.3 对象图（图 4） 11
3.4 组件图（图 5） 12
3.5 部署图（图 6） 13
3.6 用例图（图 7） 14
3.7 活动图（图 8） 15
3.8 状态图（图 9、表 7） 16
3.9 时序图（图 10） 17
3.10 通信图（图 11） 18

四、功能模块的 UI 界面设计 19
4.1 设计规范（表 8） 19
4.2 顾客端（图 12 ~ 图 19） 20
4.3 商家端（图 20 ~ 图 23） 24
4.4 配送端（图 24、图 25） 26
4.5 平台管理端（图 26、图 27） 27
4.6 界面与需求对照（表 9） 28
4.7 异常态与可用性自检（表 10） 29

五、AI 辅助设计的过程记录 30
5.1 使用方式与记录（表 11） 30
5.2 D01：架构与模块划分 30
5.3 D02：类图初稿与重画 30
5.4 D03：用例图与活动图 31
5.5 D04：状态图与需求缺口 31
5.6 D05：UI 页面与控件 31
5.7 D06：一致性与数据核对 31
5.8 大模型的能力边界 32
5.9 让人只做检查校对的工作流（表 12） 32

六、实验总结 33
6.1 收获与不足 33
6.2 与实验一的回改（表 13） 33

七、版本与提交 34
7.1 仓库与目录 34
7.2 提交记录（表 14） 34
7.3 AI 生成与人工修改的对应 34

一、项目与主要功能

1.1 项目与本次任务

沿用实验一的项目：**校园食刻——校园及周边外卖点餐与配送系统**。甲方在校内经营两家小吃店，日均约 150 单，现有微信群下单方式存在漏单、改价争议、地址含糊三个痛点。实验一已把诉求整理为 25 条需求（已确认 22、待确认 1、已拒绝 2），本次任务是把它变成可实施的设计：**一套分层架构与 12 个模块、三类共 11 张 UML 图、四个角色共 16 屏界面，以及 AI 辅助设计的完整记录**。

> **来自实验一的硬约束**手机浏览器中文界面、不做 App / 小程序（REQ-025）；单机部署、单库事务（REQ-016、REQ-022）；订单 / 支付 / 配送 / 退款 / 库存五对象状态变更须同事务（REQ-021）；日志保留 180 天且任何角色不可改删（REQ-023）。

交付物：`实验报告.pdf`、`UML/`（11 张图，每张含 `.puml` 源码与 `.png`）、`UI界面/`（16 屏）。

1.2 软件的主要功能

功能按业务闭环归为 12 组，每组恰好对应一个模块（表 2），跨组协作只走接口与事件。

功能组	要点	需求
账号与权限	手机号注册登录、四类角色绑定；请求按"角色 + 数据归属"双重校验，越权返回 403；管理员不可改金额、支付与配送状态	REQ-001、002
店铺与营业状态	店铺资料、营业时间、配送半径；营业中 / 休息中 / 暂停接单三态切换，5 秒内生效，休息中仍可浏览但不可下单	REQ-003
菜品、规格与查询	菜品、分类、规格（份量 / 辣度）；价格与库存在规格上，展示价为最低价，有历史订单只可下架；按关键词、分类、价格、有货四项组合查询并分页	REQ-004、005
购物车与快照	"用户 + 菜品 + 规格"唯一、只留同店商品；加购后改价须提示确认；提交时把名称与单价写成快照	REQ-006
下单、库存与支付	同事务校验并扣库存、固化快照、生成订单；幂等键 30 分钟；支付只认服务端回调；15 分钟未付自动关闭并释放库存	REQ-007
接单、拒单、超时	接单→制作中→出餐→待配送；拒单必选原因并全额退款；3 分钟提醒、5 分钟未接单自动取消	REQ-008
配送分配与执行	出餐生成唯一配送单，指派与抢单只生效一种；取餐→送达；失败须选原因；顾客改自取不退款	REQ-009、010
取消与退款	待支付直接关闭；待接单可自行取消；制作中及之后需商家同意；退款失败进人工队列	REQ-011
查询与统计	四类角色按可见范围查询订单，支持状态 / 时间 / 金额 / 订单号 / 手机号后四位；净营业额、订单量、销量排行、时长指标并标注数据截止时间	REQ-012、015
地址与配送范围	最多 10 个地址、默认唯一；按配送半径校验，超范围提示可选自取；手机号格式校验	REQ-020
日志与审计	17 类操作日志与业务同事务写入，字段含改前改后与 IP；只读、保留 180 天、可按订单号追溯	REQ-019、023

表 1　12 组功能与对应需求

明确不做：评价与打分（REQ-013）、优惠券与积分（REQ-014）、部分退款、发票、骑手实时轨迹、预约单、App / 小程序、多语言。因此界面中不出现评分、优惠券入口，模块划分里也没有对应模块。

二、软件架构设计

2.1 架构风格与理由

三条约束决定了选型：核心业务必须落在单库事务里（REQ-021），部署是单机双实例（REQ-016），业务规则密集在订单生命周期上。据此选择**分层单体 + 领域模块 + 事件解耦**：对外是一个无状态 Web 应用，对内按表现 / 应用 / 领域 / 基础设施四层组织，层内按业务切包；强一致协作走同步接口与本地事务，弱一致协作走消息与延迟任务。

不用微服务：12 个模块共享同一份订单数据，"扣库存并固化快照"拆开就要引入补偿事务，与"整单失败并回滚"（REQ-007）冲突。不用纯 MVC 三层：状态机、幂等、审计会被复制到各 Service，正是实验一担心的"多处描述互相矛盾"。代价是模块边界靠包依赖规则约束，因此把规则写成可检查的约定（2.2）。

2.2 分层与依赖规则

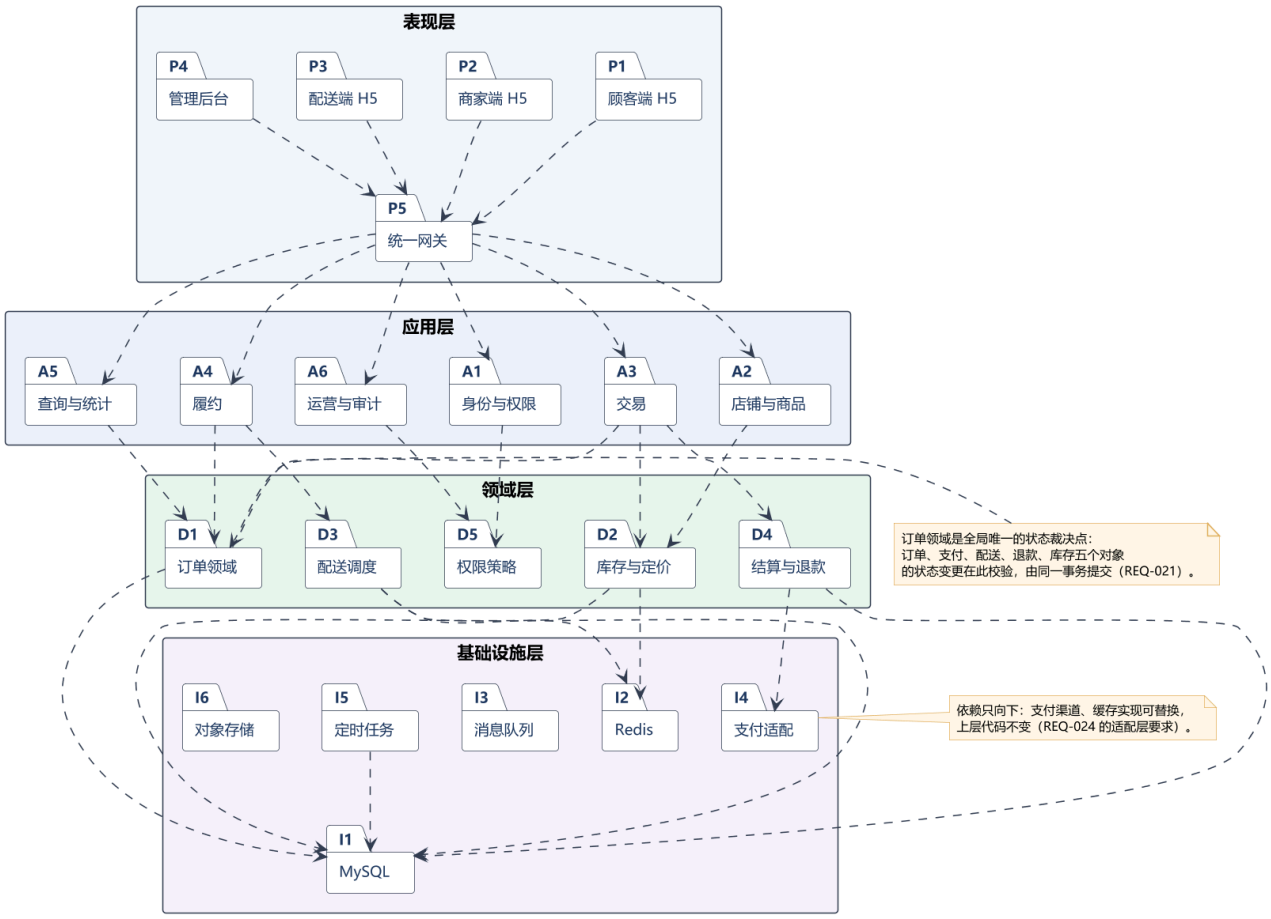


图 1　分层架构包图（结构性：包图）

依赖规则四条：**只向下依赖**（表现层不直连基础设施）；**领域层通过端口接口访问外部资源**，基础设施实现接口，因此换支付渠道或缓存不改上层代码（REQ-024）；**跨模块只走对方应用服务接口**，不引用对方实体；**订单领域是唯一状态裁决点**，其他模块不得自行修改订单状态。

2.3 模块划分

编号	模块	职责	关键类 / 组件	需求
M1	账号与权限	注册登录、验证码与锁定、角色绑定、凭证失效、双重鉴权、账号停用	账号、四类角色、权限策略	001、002、017
M2	店铺与营业状态	店铺资料、营业时间、三态切换、配送半径、状态生效	店铺、营业状态机	003
M3	菜品与库存	菜品分类、规格价格、库存扣减与释放、上下架与售罄、只可下架规则	菜品、规格、库存服务	004、005、007
M4	购物车与订单	购物车同店约束、改价比对、下单事务、幂等键、订单查询与详情	购物车项、订单、订单明细、订单状态机	006、007、012
M5	支付与退款	支付单、渠道适配、回调验签与幂等、退款单与回执、超时退款、对账	支付单、退款单、支付适配层	007、011、022、024
M6	商家履约	接单、拒单（必选原因）、出餐、3 / 5 分钟提醒与自动取消、拒单率	履约服务、延迟消息	008
M7	配送	配送单生成、指派与抢单、取餐、送达、失败、改自取、到店超时	配送单、派单服务、抢单池	009、010
M8	收货地址	地址增删改、默认唯一、10 条上限、手机号校验、距离与范围判定	收货地址、距离计算	020
M9	查询与统计	多条件查询与分页、统计口径、数据截止时间、从库只读	统计服务、只读视图	012、015
M10	通知与提醒	站内、短信、订阅消息；支付成功、倒计时、到店超时、退款结果	通知服务、消息消费者	008、009、010
M11	日志与审计	17 类日志同事务写入、只追加只读、按订单追溯、180 天归档	操作日志、审计组件	019、023
M12	平台管理	账号管理、店铺审核与下架、违规处理、全平台统计、订单作废（不可改金额）	管理后台、运营服务	002、019、023

表 2　模块划分与职责（M1~M12 覆盖实验一全部已确认需求）

另有不计编号的**公共能力**：统一错误码（403 / 409 语义固定）、幂等键组件（Redis + 唯一索引）、事务模板（同事务写日志）、距离计算、分页规范。抽出来是为了让"越权不泄露""日志缺失即失败"这类规则只有一处实现。

2.4 需求覆盖

需求	模块	UML	界面
REQ-001 注册登录	M1	图 3、7	登录按通用表单实现，未占 16 屏配额
REQ-002 角色与权限	M1、M12	图 3、7	图 27（越权尝试日志）
REQ-003 店铺与营业状态	M2	图 2、5	图 12、20
REQ-004 菜品与规格	M3	图 2、3	图 22
REQ-005 菜品查询与筛选	M3、M9	图 7	图 12
REQ-006 购物车与快照	M4	图 2、4	图 13、14
REQ-007 下单支付	M4、M5	图 8、10	图 15、16
REQ-008 接单拒单超时	M6、M10	图 9、10	图 20、21
REQ-009 配送分配	M7	图 11	图 24

需求	模块	UML	界面
REQ-010 配送执行	M7	图 9、11	图 25
REQ-011 取消与退款	M5	图 9	图 17、18
REQ-012 订单查询详情	M4、M9	图 2	图 17、18
REQ-013 评价管理（已拒绝）	—	—	不设入口
REQ-014 优惠券积分（已拒绝）	—	—	不设入口
REQ-015 经营统计	M9	图 5	图 23、26
REQ-016 查询性能	M9	图 6	设计约束
REQ-017 账号与数据安全	M1、M8	图 3	图 18、25（脱敏与可见性）
REQ-018 下单易用性	M4	图 8	图 12~16（四页面）
REQ-019 操作日志	M11	图 5	图 27
REQ-020 地址与范围	M8	图 3、7	图 19
REQ-021 状态机	M4	图 9	图 16~18
REQ-022 数据一致性	M4、M5	图 10	设计约束
REQ-023 保留与审计	M11	图 5、6	图 27
REQ-024 支付渠道约束	M5	图 1、6	图 15（沙箱标识）
REQ-025 客户端形态约束	表现层	图 1、6	全部 16 屏（390px 宽）

表 3　25 条需求到模块、UML 与界面的覆盖

2.5 模块之间的交互

场景	发起→接收	方式	一致性要求与失败处理
下单扣库存	M4→M3	同步 + 同事务	规格行加锁；任一不足则整单回滚并返回缺货项，购物车保留
地址与营业校验	M4→M8、M2	同步	先算距离再判半径与营业状态；超范围拒绝并给自取路径
支付与回调	M4→M5→支付平台	同步 + 异步回调	验签与金额比对通过才改状态；重复回调按渠道交易号幂等
支付成功通知	M4→M10	消息（outbox）	先写 outbox 再投递，消费幂等；通知失败不阻断状态流转
待接单超时	M6→M4、M5	延迟消息	3 分钟提醒、5 分钟取消退款；与接单并发以行锁先落库者为准
出餐生成配送单	M4→M7	同事务 + 事件	配送单与订单状态同事务生成，单号唯一并进入抢单池
指派 / 抢单	M6、M7→M7	同步 + 行锁	行锁 + 抢单池二次校验，只允许一人成功，另一人 409
送达回写	M7→M4	同步	写时间戳并触发"配送中→已完成"，供统计使用
取消与退款	M4、M6→M5	同事务 + 异步	取消与退款单同事务；回执异步到达后转"已退款"
写操作日志	全部→M11	同事务	业务成功而日志缺失视为失败；日志只追加
统计查询	M9→M4、M5	只读从库	允许秒级延迟，返回数据截止时间，不参与写路径
对账告警	M5→M7、M11	定时任务	每 10 分钟比对；差异只允许补偿任务修正

表 4　模块交互方式、事务边界与失败处理

三条主链路值得单独点出：**下单支付**（M4 校验 → M3 扣库存 → M4 写单与快照 → M5 回调验签 → M10 通知）、**接单配送**（M6 计时 → M7 生成并分配配送单 → M7 回写 M4 状态）、**退款对账**（四条取消

路径都落到退款单 → 回执 → 已退款，失败进人工队列）。前两条要求强一致，第三条允许最终一致但必须可对账。

2.6 关键设计决策

问题	选择	代价
状态散落	8 个状态、15 条转换收敛到订单领域，其他模块只提交事件	变更须经一层间接调用，由"流水必写"兜底
改价导致口径漂移	提交时把菜品名、规格名、单价写成快照，之后不再连表取价	数据冗余；历史订单显示旧名称（正是凭证语义）
网络重试变两单	幂等键：Redis 判重 + 数据库唯一索引兜底，命中返回首次订单	需键清理与容量评估，且要处理"键同但购物车已变"
超卖与抢单冲突	以数据库行锁为最终裁决，Redis 只做前置过滤	吞吐受单行锁限制（峰值 30 QPS 未触及）
审计缺失	日志与业务同库同事务，业务成功而日志缺失视为失败	日志增长快，需 180 天归档并保证可检索
三类超时	延迟消息触发，定时对账兜底	消息重复投递，消费端按"业务号 + 事件"幂等
渠道切换	支付端口 + 沙箱 / 微信支付两套实现，只改配置	多一层抽象
统计拖慢交易库	统计走从库，界面固定标注数据截止时间	用户可能看到"刚下的单未计入"，需界面解释口径

表 5 关键设计决策：问题、选择与代价

三、UML 建模

3.1 三类图与本项目的选型

先列每类图能回答的问题，再判断本项目是否需要。表 6 是结果：结构性 5 张、行为性 3 张、交互性 2 张，另有 4 种图未采用并说明理由。全部用 PlantUML 绘制，`.puml` 与 `.png` 一并提交（150 dpi，中文字体 Microsoft YaHei）。用文本源码而非拖拽，是因为类图与活动图都重画过（见 5.3、5.4），源码让"改一处、看一遍"的成本极低。

类别	图	能回答的问题	本项目	理由
结构性	类图	有哪些类、属性方法、类间关系	采用 2 张	必画图；领域对象与关系是建表与接口的基础
	对象图	某时刻一组实例及其取值	采用	类图说不清"改价后旧订单仍按旧价结算"，实例快照最直观
	组件图	可替换部件及其接口协作	采用	表达"只能通过接口调用"的依赖规则，也是将来拆服务的切割线
	部署图	软件运行在哪些节点、如何通信	采用	性能目标与"双实例无状态 + 定时任务选主"须落到节点才可验收
	包图	代码分层与依赖方向	采用	比组件图更适合表达"四层 + 只向下"
	复合结构图	类内部部件、端口与连接器	未采用	本项目组合关系简单（订单—明细），展开端口是为画而画
	剖面图	在某个关注点上特化模型	未采用	规模不足以体现价值，权限约束改用类图注记
行为性	用例图	谁能做什么、系统边界在哪	采用	把 25 条需求收敛为 20 个用例，并显式标出两个系统参与者
	活动图	业务流程的控制流与分支	采用	下单—支付最关键，需把三条异常分支画进同一张图
	状态图	对象生命周期内的状态迁移	采用	REQ-021 本身就是状态机约束，状态图是其图形化表达
交互性	时序图	对象按时间顺序交换的消息	采用	表达"验签—幂等—同事务改状态"的顺序最清楚
	通信图	对象链接与带编号的消息	采用	并发抢单的重点是"两事务争同一行"，把链接放在前景更能说明锁
	序列图	与"时序图"同类，译名不同	同图 10	统一使用"时序图"名称，不重复绘制
	交互概览图	多个交互串成控制流	未采用	编排已由活动图表达，重复绘制会造成两张图口径不一致

表 6　三类 UML 图的用途与选型（采用 11 张，未采用 4 张并说明理由）

3.2 类图

类图分两张画：第一版一张画了 18 个类、1900px 宽，打印后字号不足 4pt，且连线交叉无法阅读（见 5.3）。图 2 是交易与履约域，回答"一次订单由哪些对象构成"；图 3 是账号与商品域，回答"谁能下单、能点哪些菜"。

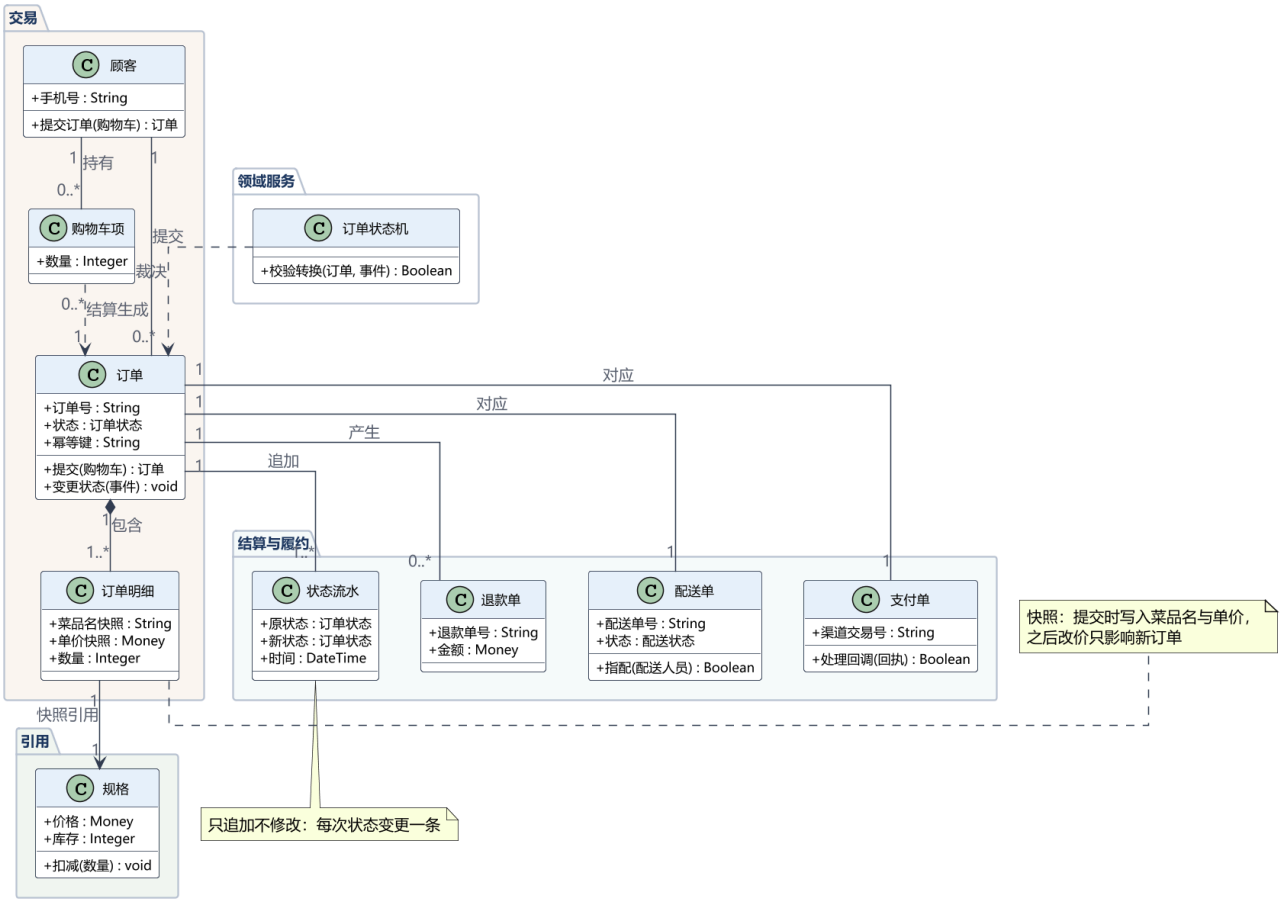


图 2　类图（交易与履约域）：订单、明细、支付、配送、退款与状态流水

四个要点：**组合与关联是刻意区分的**——订单与明细为组合（明细不能脱离订单），订单与支付单、配送单为 1:1 关联（支付单在订单取消后仍要参与退款，生命周期不同步）；**明细到规格是依赖而非组合**，明细只存快照值，规格改价不影响它；**状态流水是只追加实体**，而不是订单表上的时间字段，才能满足"每次变更写一条、不可修改"；**订单状态机作为领域服务出现**，说明"能不能变"由转换表裁决，而不是各写一套 if。

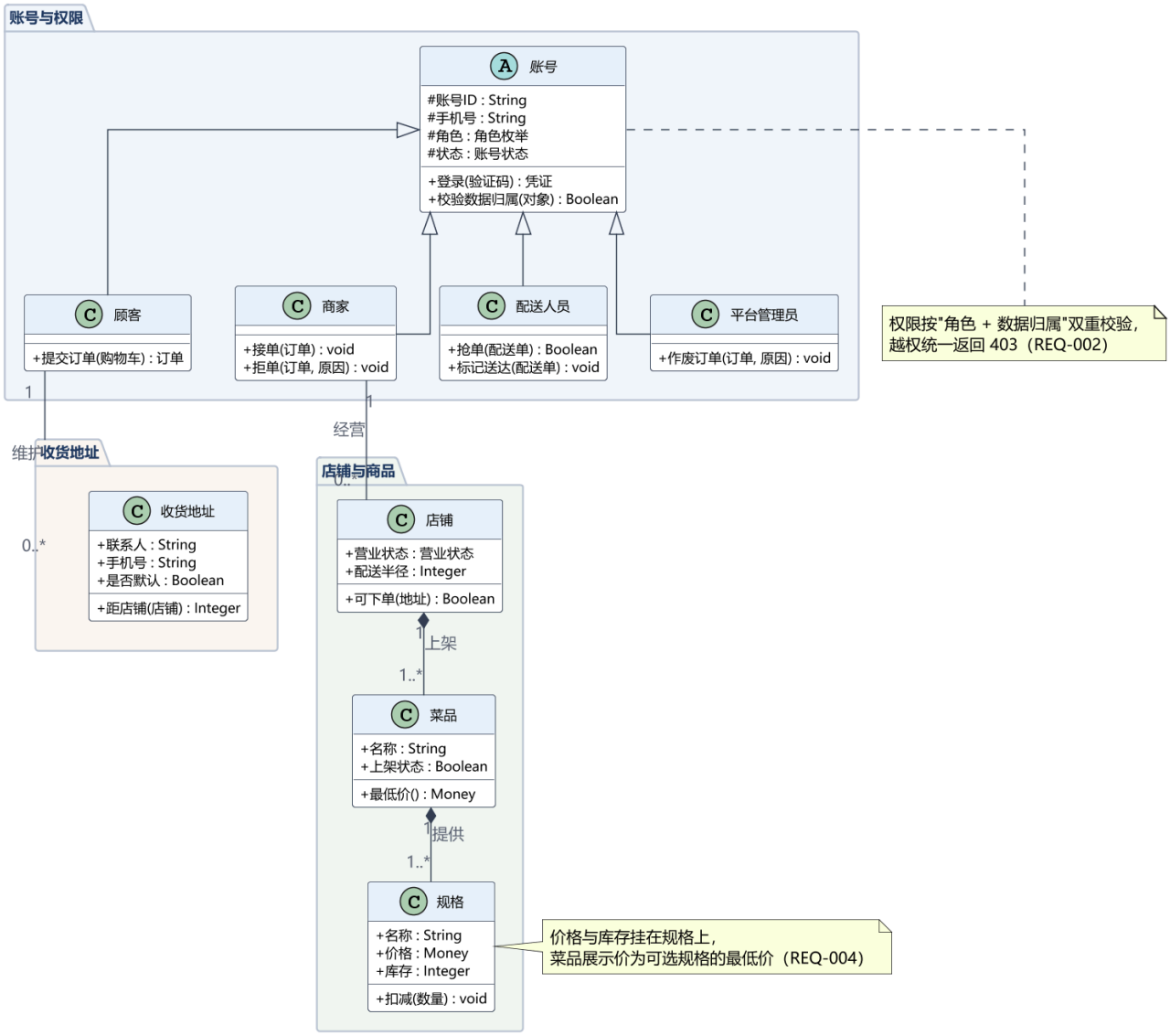


图 3　类图（账号与商品域）：角色继承体系、店铺—菜品—规格与收货地址

角色用继承（账号 → 顾客 / 商家 / 配送人员 / 管理员）而非角色枚举，因为四类角色行为差异明显；用继承后越权在类型层面就不成立。两个易漏点：商家与店铺是 1:N，营业状态必须挂在店铺上（REQ-003 要求按店独立）；订单不直接关联收货地址，只存地址快照，因为地址可被删除修改而订单必须永远能显示"送到哪"。

3.3 对象图

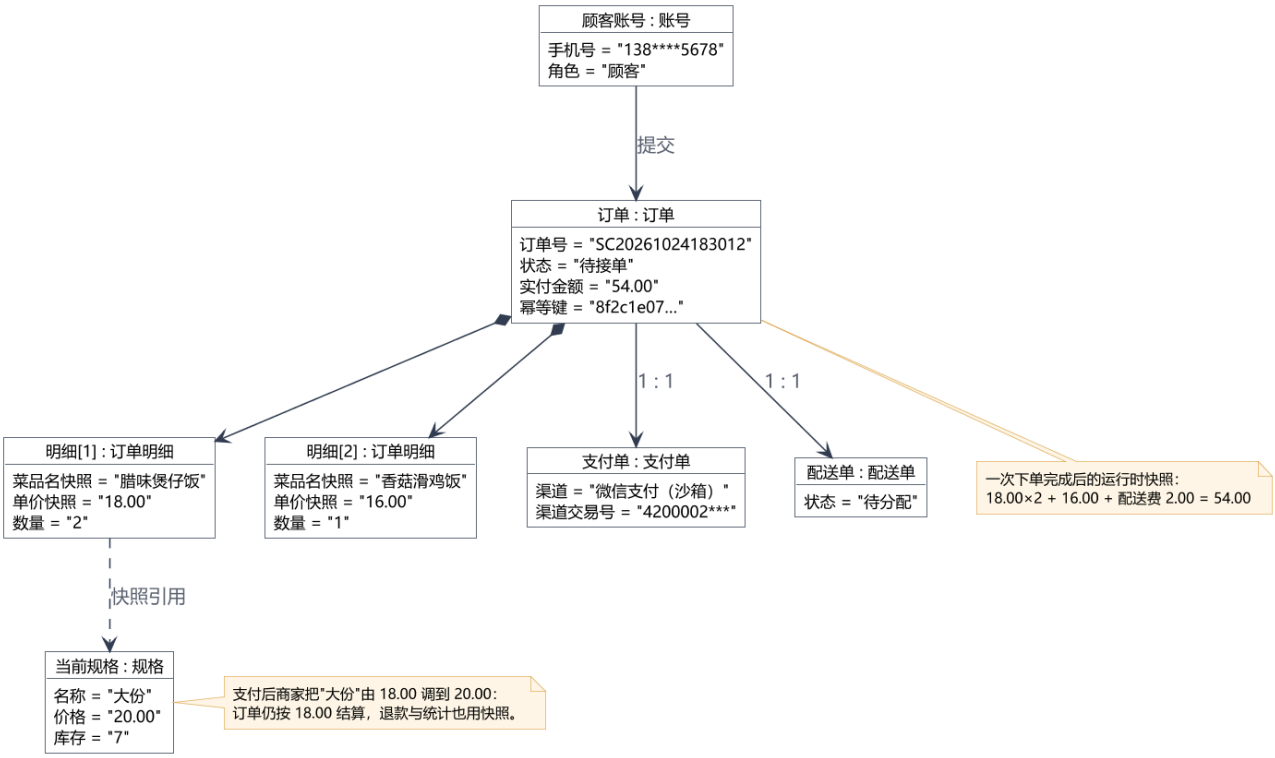


图 4　对象图：改价之后的订单实例，明细仍保留旧价

图 4 是一次下单支付后的实例快照：明细单价快照为 18.00，而当前规格价格已是 20.00。它直接对应 REQ-006 的验收场景——结算页外改价后返回，应出现价格变动提示；同时说明退款与统计都按 18.00 结算，历史报表不会因一次改价而漂移。只画类图时这个语义只能靠文字描述，实现时极易被写成"连表查当前价"。

3.4 组件图

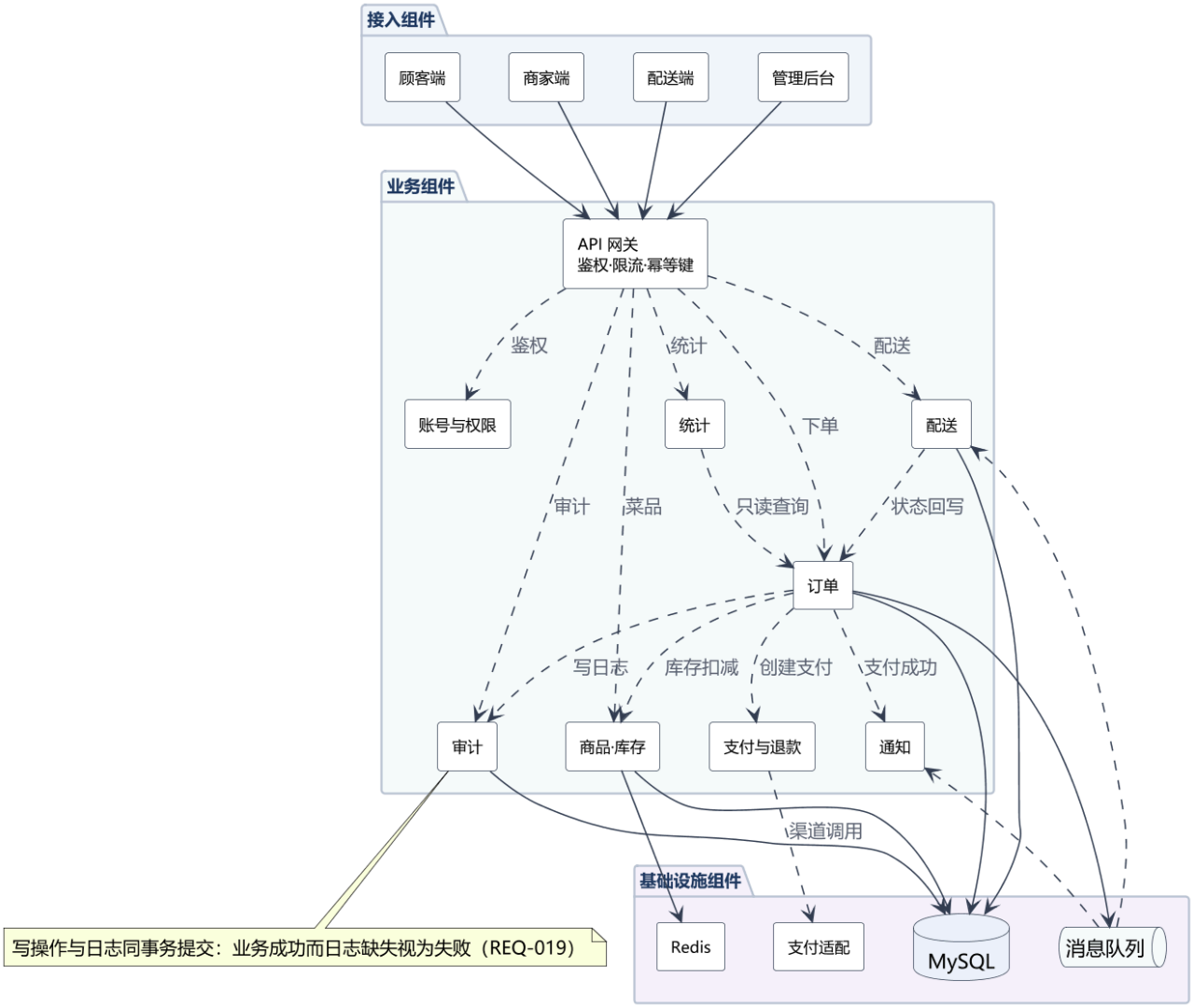


图 5　组件图：接入、业务与基础设施组件及其接口依赖

三个约束：四个端只连网关，鉴权、限流、幂等键注入只有一处实现；业务组件之间是接口依赖而非数据依赖，订单组件需要"扣库存"但不关心库存如何存储；审计组件被所有写操作依赖，图上无法表达"同事务"，用注记补充。这也说明 UML 图不是万能的，关键约束仍需文字兜底。

3.5 部署图

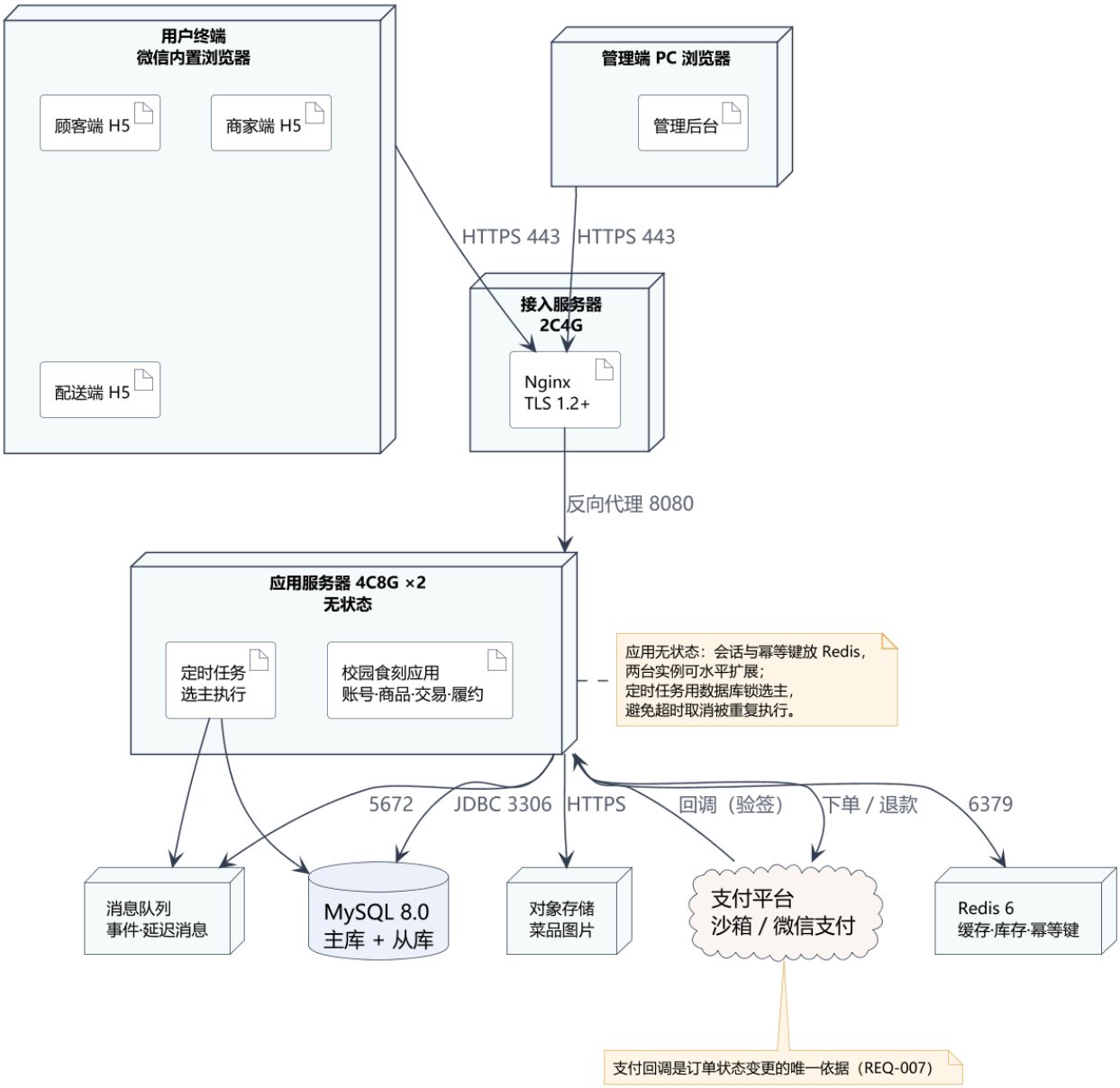


图 6　部署图：单机双实例、主从库与外部支付平台

两个刻意设计：**应用无状态**（会话、幂等键、热点库存放 Redis），因此两台实例可水平扩展，加机器即可验证 200 并发下的 95 分位指标；**定时任务用数据库锁选主**，否则 15 分钟未支付关闭会被两台实例各执行一次，虽幂等但造成无谓锁竞争与日志噪音。

3.6 用例图

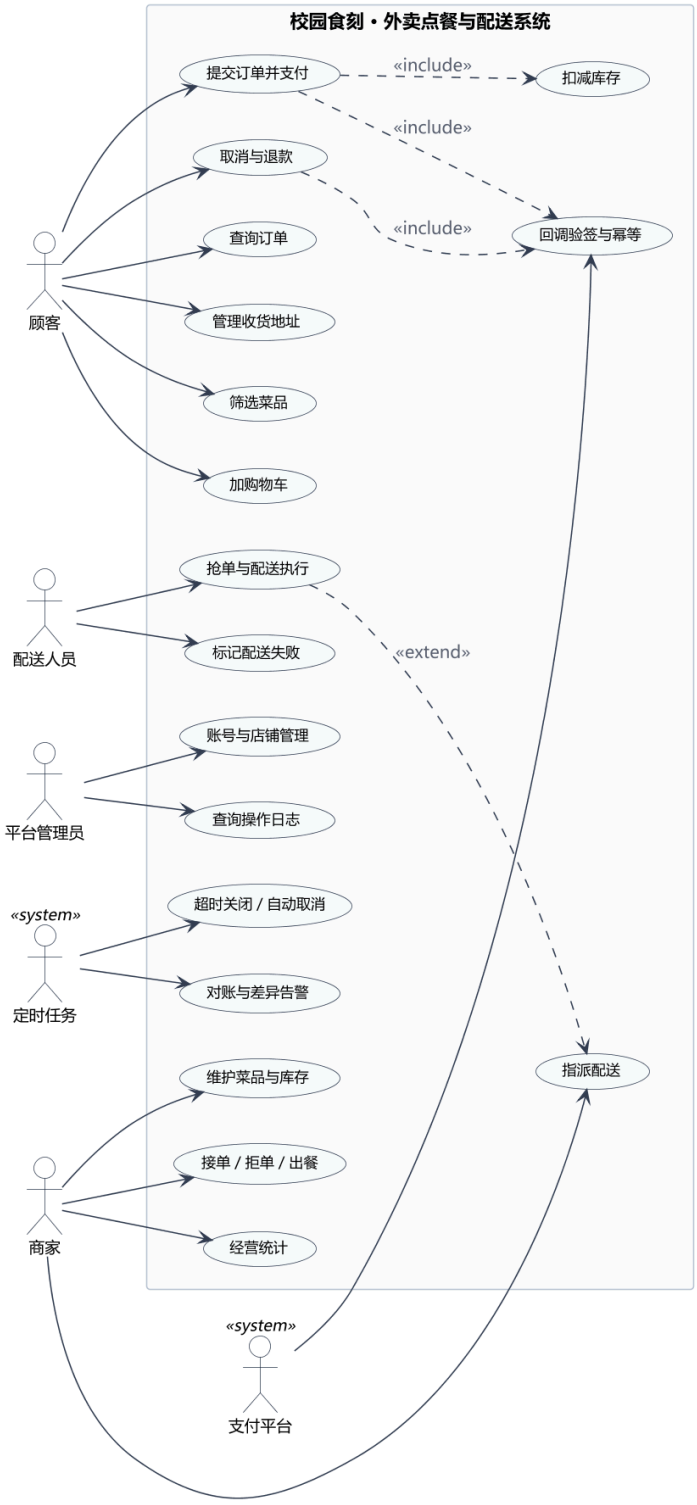


图 7　用例图：四类人类角色与两个系统参与者

把支付平台与定时任务画成参与者而非藏在流程里，是因为它们承担的动作都有验收标准（REQ-007、008、022），需要有归属。三处 `<<include>>` 表示必做步骤（扣库存、回调验签、退款验签），一处 `<<extend>>` 表示可选路径（骑手抢单相对商家指派）——把两者用反是这类图最常见的错误。

3.7 活动图

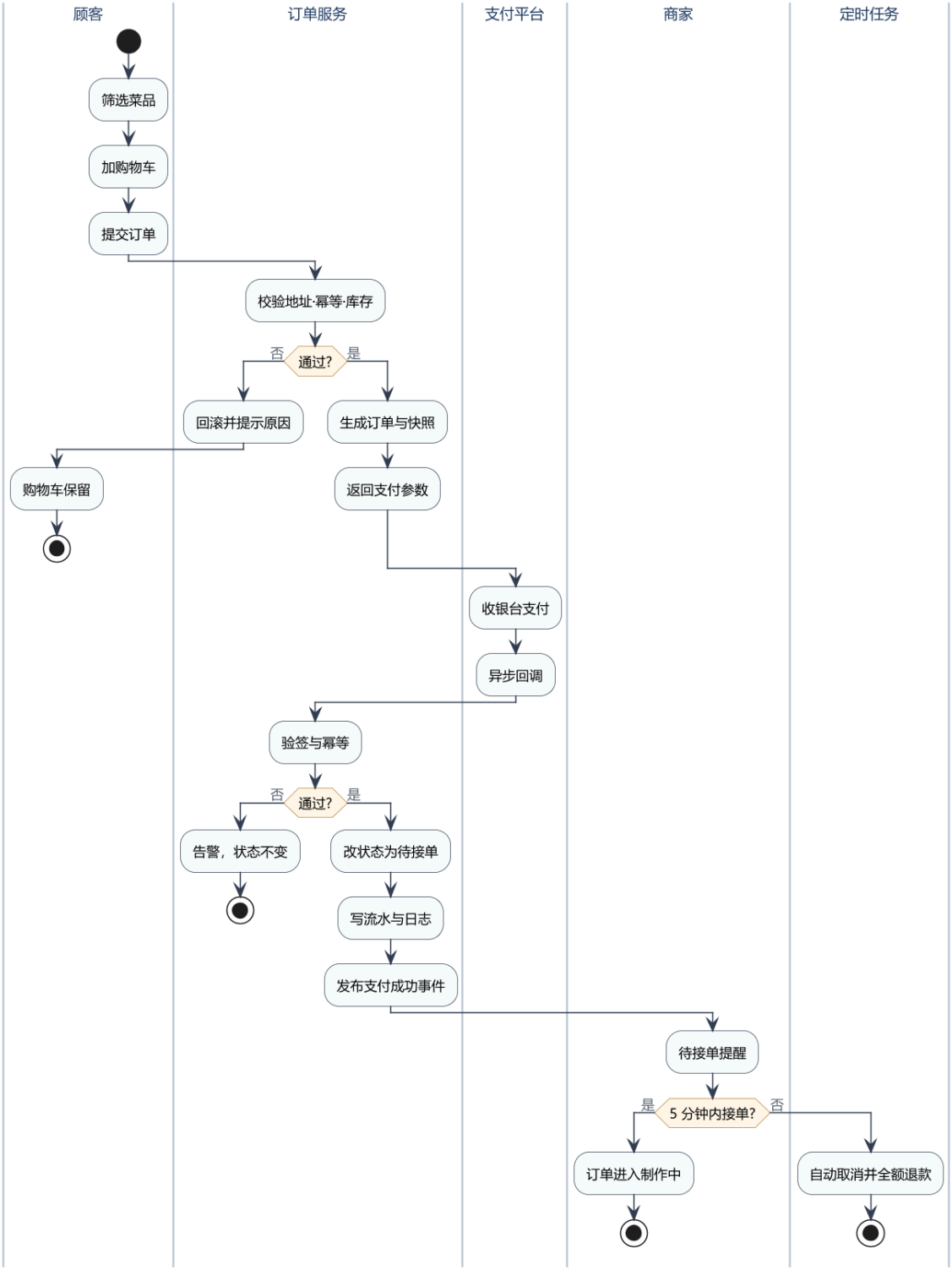


图 8　活动图：下单与支付主流程及异常分支

五个泳道覆盖从选菜到接单，三条异常分支（校验不通过、验签失败、5 分钟未接单）收在同一张图里。第一版把地址、幂等、库存画成三个并列判定，读者看不出它们共用一次事务；改后合并为"校验地址·幂等·库存"一个动作，用注记说明幂等命中时直接返回首次订单。合并让粒度变粗，但换来了事务边界的准确——活动图强调职责归属，不追求每个 if 都出现。

3.8 状态图

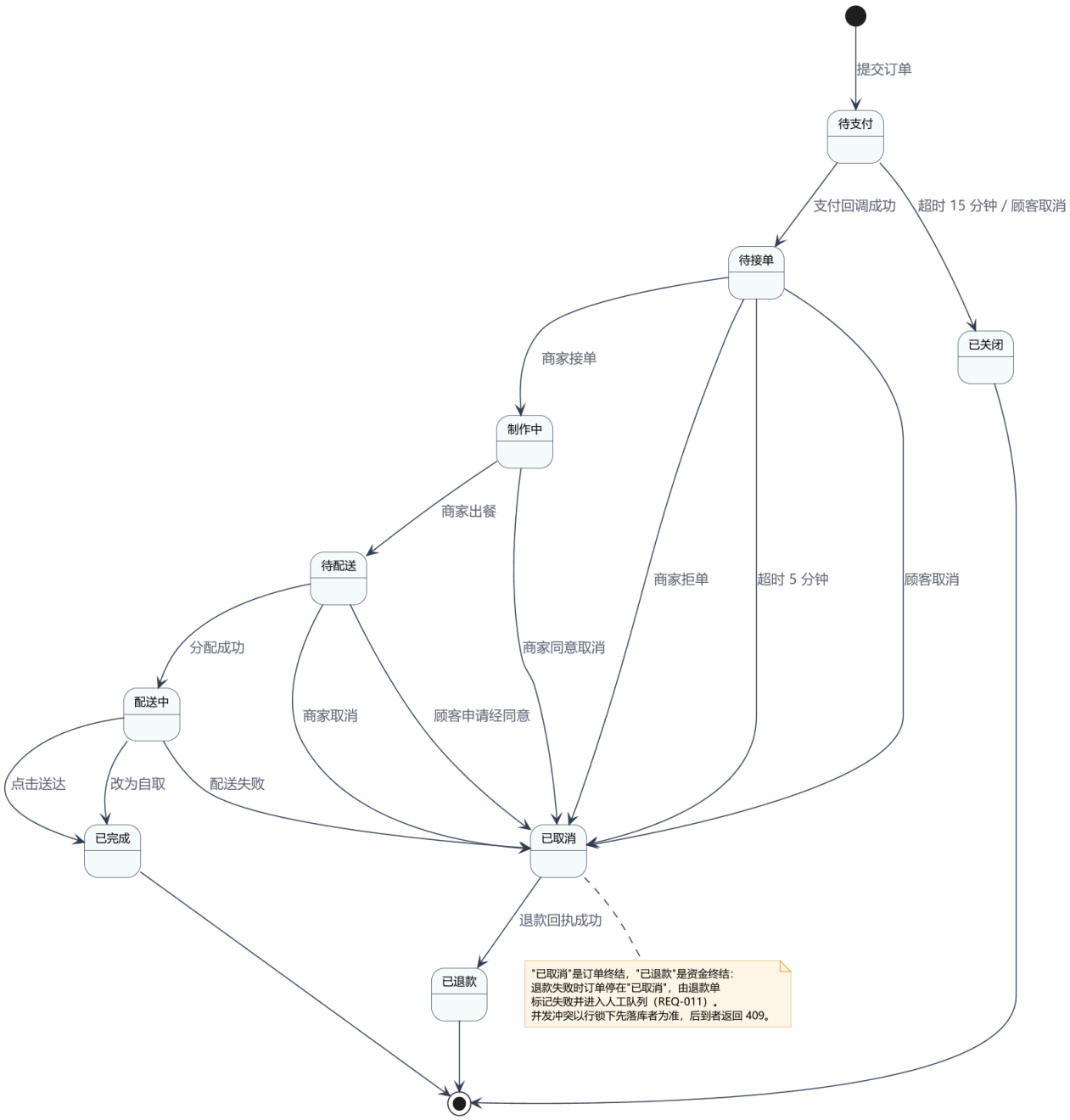


图 9　状态图：订单的 9 个状态与 15 条转换

这张图发现了一个需求缺口：原 REQ-021 的 13 条转换中，"待配送"只有"分配成功"一条出口，而出餐后无人接单的情形按 REQ-011 应可取消，两者不一致会让订单永久停在待配送。据此补了两条转换并回改实验一（表 7 第 11、12 条，详见 6.2）。

序号	当前	事件	目标	角色	说明
1	待支付	支付回调成功	待接单	系统	验签与金额校验通过
2	待支付	超时 15 分钟	已关闭	系统	同时释放库存
3	待支付	顾客取消	已关闭	顾客	未支付无退款
4	待接单	商家接单	制作中	商家	记录接单时间
5	待接单	商家拒单（必选原因）	已取消	商家	自动全额退款，计入拒单率

序号	当前	事件	目标	角色	说明
6	待接单	超时 5 分钟	已取消	系统	与接单并发以先落库者为准
7	待接单	顾客取消	已取消	顾客	全额退款
8	制作中	商家出餐	待配送	商家	记录出餐时间并生成配送单
9	制作中	商家同意取消	已取消	商家	拒绝则保持制作中
10	待配送	分配成功	配送中	商家 / 配送	指派与抢单只生效一种
11	待配送	商家取消（无人可派）	已取消	商家	本次新增，补死状态出口
12	待配送	顾客申请经商家同意	已取消	商家	本次新增，对应 REQ-011
13	配送中	骑手送达	已完成	配送	记录送达时间
14	配送中	配送失败（必选原因）	已取消	配送	全额退款，原因进统计
15	已取消	退款回执成功	已退款	系统	失败则停留已取消并进人工队列

表 7　订单状态转换规则（第 11、12 条为实验二新增）

3.9 时序图

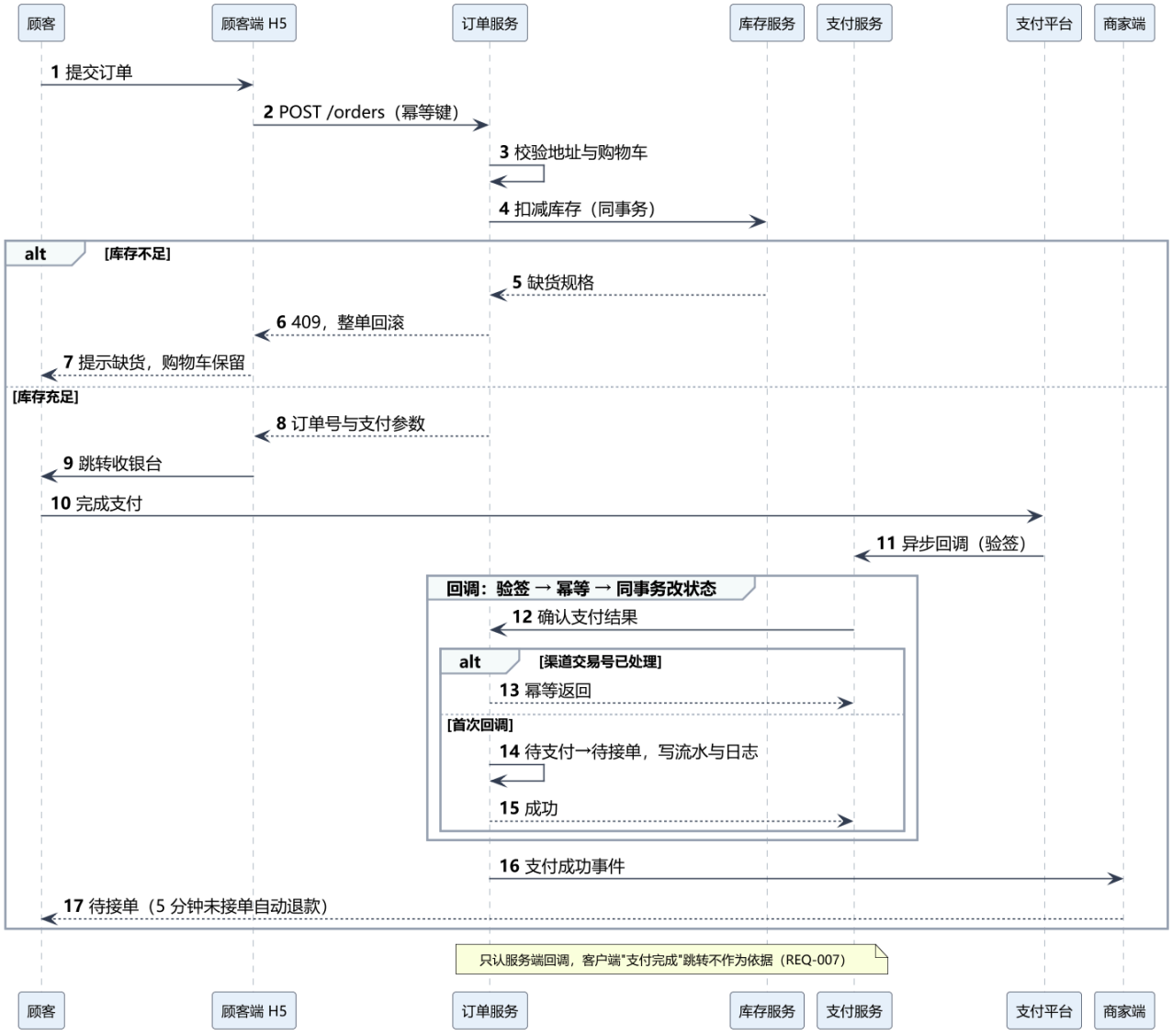


图 10　时序图：下单、支付与回调验签

三条规则：**支付结果只认服务端回调**，客户端"支付完成"跳转不产生任何状态变更；**回调处理顺序不能调换**（先验签防伪造、再幂等防重复、最后写状态），图中用分组圈出；**库存不足在支付之前失败**，因此不存在"付了钱却没货"，退款链路只服务于取消与失败场景。

3.10 通信图

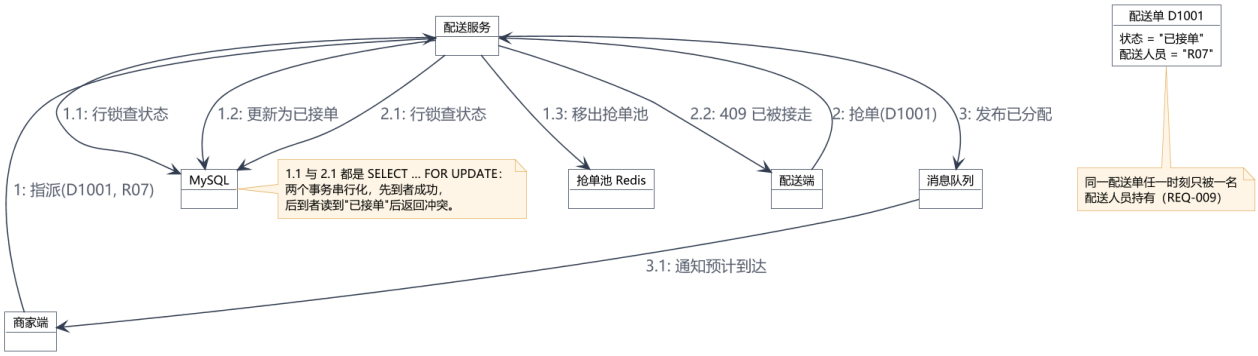


图 11　通信图：指派与抢单并发时，行锁如何保证唯一所有者

消息 1.1 与 2.1 都是 `SELECT … FOR UPDATE`，两事务串行化：先到者改状态并移出抢单池，后到者读到"已接单"返回 409。抢单池放 Redis 只为挡住无效请求，**最终裁决必须是数据库行锁**，否则缓存与库不一致时会出现两名骑手都认为自己抢到。PlantUML 没有原生通信图语法，这里用"对象框 + 层级编号箭头"手工建模，编号遵循 1、1.1、2、2.1 约定。

四、功能模块的 UI 界面设计

4.1 设计规范

先定规范再画，否则 16 屏会变成 16 套配色与 16 种按钮尺寸。表 8 是本次使用的规范，每一条都对应需求中的具体约束。

项目	规范	依据
画板	移动端 390×844（2 倍图 780×1688）；桌面端 1440×900	覆盖主流手机；2 倍图保证 PDF 放大不虚（REQ-025）
色彩	品牌橙 `#FF6B2C`；成功 `#12B76A`、警示 `#F79009`、危险 `#F04438`、进行中 `#2E90FA`；正文 `#1D2433`、次要 `#6B7280`、页面底 `#F5F6F8`	状态固定用色：待处理橙、进行中蓝、成功绿、失败红
字号	金额 19–22、标题 17–19、正文 13–15、辅助 11–12	金额最大，其次商品名，说明最小
触摸目标	可点击元素 ≥ 44×44，按钮高度 38–46	REQ-018 明确要求
栅格	8pt 栅格，页边距 14–16，卡片间距 10–12，圆角 14–16	控制一屏信息密度
脱敏	手机号 `138****5678`；完整地址仅"已接单—配送中"对骑手可见	REQ-017 的可见性规则写进界面
页面数	主流程固定四页：店铺 → 购物车 → 确认订单 → 支付结果	REQ-018 要求不超过 4 个页面
异常态	售罄置灰不可选、超范围红色提示、倒计时变红、空态给下一步动作	见 4.7 自检表

表 8　UI 设计规范

关于工具：这 16 屏不是 Figma / Pixso 拖拽的，而是用自写的 HTML/CSS 组件库（卡片、标签、步进器、分段控件、弹层等 20 余个组件）编写，再用无头 Chromium 以 2 倍像素密度导出。理由有三：后续实现与设计稿同源，避免两张皮；16 屏共用一份样式表，改主色一次生效；稿子能进 Git，"生成 → 修改"的 diff 可查。不足是缺少设计工具的组件面板与协作评论，图标需手写 SVG（见 5.6）。

4.2 顾客端

顾客端 8 屏，覆盖"看菜 → 选规格 → 购物车 → 确认订单 → 支付 → 查订单 → 管地址"。



图 12　顾客-01 店铺与菜品筛选



图 13　顾客-02 规格选择

图 12 首屏同时承担"决策"与"浏览"：店铺信息卡给出营业状态、月售、起送、配送费、预计时长与公告——营业状态必须在第一屏，否则用户只会觉得"点了没反应"（REQ-003）；下面依次是搜索框与四项筛选、菜品行（缩略图、简介、"¥18 起"、加号），售罄行置灰并换成不可点击的"售罄"。**图 13** 用半屏弹层而非新页面，不打断浏览；份量中"小份"标售罄并禁用，辣度标必选，底部实时显示合计，把"价格挂在规格上"与"单行上限 99"做成可见约束。



图 14　顾客-03 购物车



图 15　顾客-04 确认订单

图 14 重点在三个异常态：橙色提示条写明"已由 ¥18.00 上调为 ¥20.00，确认后按新价结算"，行内保留删除线原价（REQ-006）；售罄商品整行置灰、复选框不可点、数量禁用；底部合计只统计可售商品，并用小字说明"数量减到 0 即移除""加入其他店铺前需先结算"。**图 15** 按"地址 → 方式 → 商品 → 备注 → 支付"排列：地址卡给出"距店铺 1.2km，在配送范围（3.0km）内"的结论（距离由后端算，前端只呈现）、配送方式可切到店自取、商品逐行标注单价×数量、合计 58.00（商品 56.00 + 配送费 2.00）、支付方式标注"测试环境，不会真实扣款"。



图 16　顾客-05 支付结果



图 17　顾客-06 订单列表

图 16 同时回答"钱付没付"和"饭何时来"：沙箱标识避免误认为真实扣款（REQ-024）；待接单卡片给 4 分 32 秒倒计时并写明"5 分钟未接单自动取消并全额退款"，把超时规则前置给用户；末尾一行小字"以支付平台服务端回调为准，页面跳转不代表支付成功"。**图 17** 按状态分页签，订单卡按状态给不同主操作（联系商家 / 申请退款、查看进度、再来一单），底部提示"仅展示近 90 天订单"（REQ-012）。



图 18　顾客-07 订单详情



图 19　顾客-08 收货地址

图 18 顶部是状态词、预计送达与六段进度条，往下是骑手卡（脱敏手机号）、地址卡、明细（标注"金额为下单时的价格快照"）与底部操作；卡片上写明"配送完成后地址将不可见"，让用户理解信息什么时候对谁可见（REQ-017）。**图 19** 用三张卡展示默认、普通、超范围三种状态，超范围行给"距店铺 3.2km，超出配送范围（3.0km）…可选择到店自取"，页脚三条小字说明门牌缺失、手机号校验与 10 条上限。

4.3 商家端



图 20　商家-01 接单工作台



图 21　商家-02 拒单原因（必选）

图 20 只做一件事：让商家 5 分钟内处理订单。顶部营业状态三档切换并标注"顾客端 5 秒内生效"，下面三个数字强调待接单，再往下按状态排三类订单卡（接单 / 拒单、出餐、指派骑手 / 撤回）。**图 21** 是拒单弹层：四类原因默认选中第一项，未选时"确认"不可点，并说明"拒单后立即取消并全额退款，原因计入拒单率"——REQ-008 与 REQ-011 的界面证据。



图 22　商家-03 菜品与规格管理



图 23　商家-04 经营统计

图 22 每行展示各规格价格与库存、上架开关与售罄标签；"梅菜肉饼饭"标"有历史订单，仅可下架"，界面上根本不提供删除按钮（REQ-004）；底部提示条重申最低价规则与快照语义。**图 23** 顶部固定"数据截止 12:00（每小时刷新）"，主体是净营业额与 7 日柱状图，下接订单量、退款、取消率三张卡，再往下是销量排行与履约时效，页脚写明两个时长指标的计算口径——指标必须带口径，否则同一报表会算出不同结果。

4.4 配送端



图 24　配送-01 抢单池与我的任务



图 25　配送-02 配送执行

图 24 顶部显示在岗状态、今日完成单量与收入，三个页签区分可抢单 / 进行中 / 已完成；抢单卡给出预计收入、距离、取送点与取餐码，第二张卡演示抢单失败后按钮变为"该单已被接走"（REQ-009）。**图 25** 是作业界面：路线示意、取餐码、商家与顾客信息、四个分级操作按钮（取餐为橙色主操作、送达为绿色、配送失败为红色描边），并提示"完整地址仅在配送期间可见，送达 1 小时后自动不可见"（REQ-017）。

4.5 平台管理端

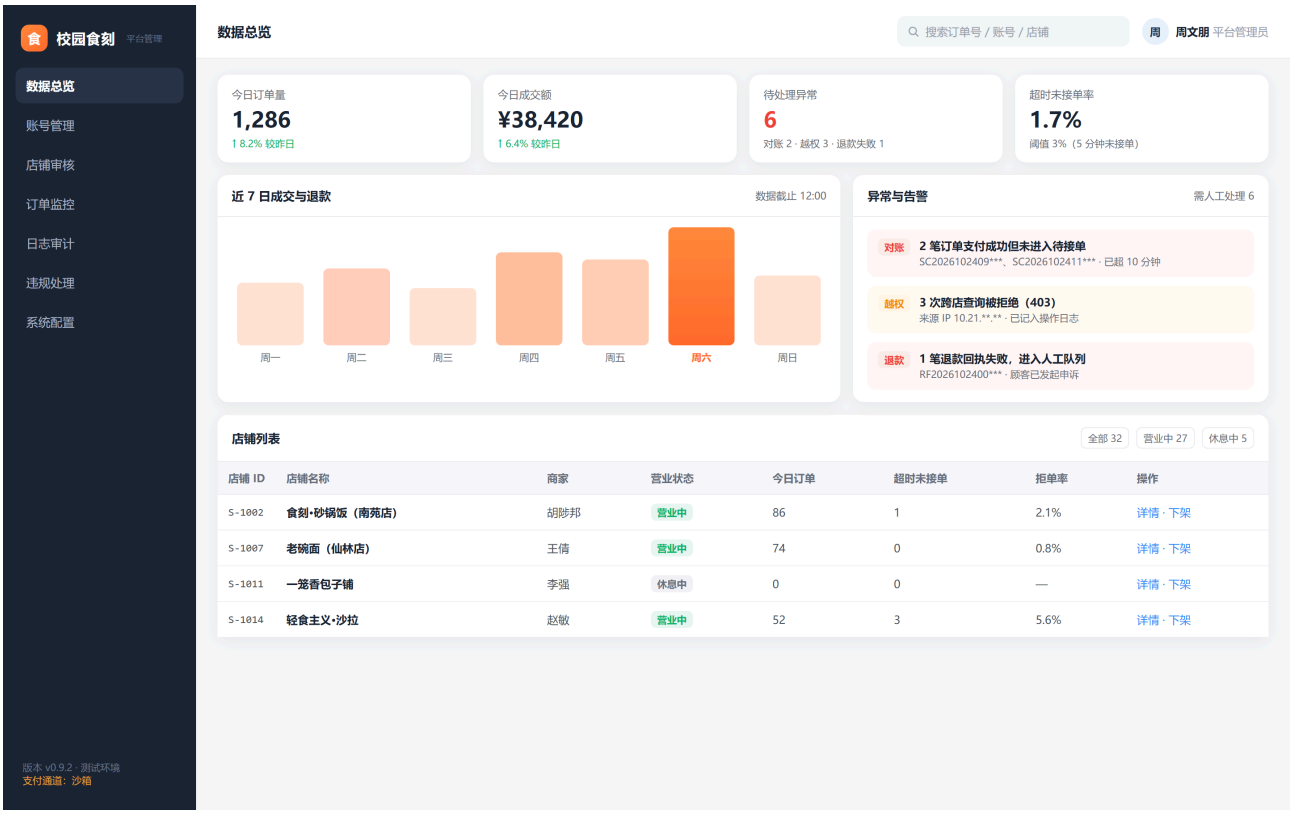


图 26　平台-01 数据总览

管理端是唯一在 PC 上使用的端。图 26 自上而下是四个指标卡、近 7 日成交柱状图与"异常与告警"（对账差异、越权、退款失败）、店铺列表。设计意图是让管理员先看"哪里出问题"，再进具体店铺，而不是一上来面对一堆可编辑表格。

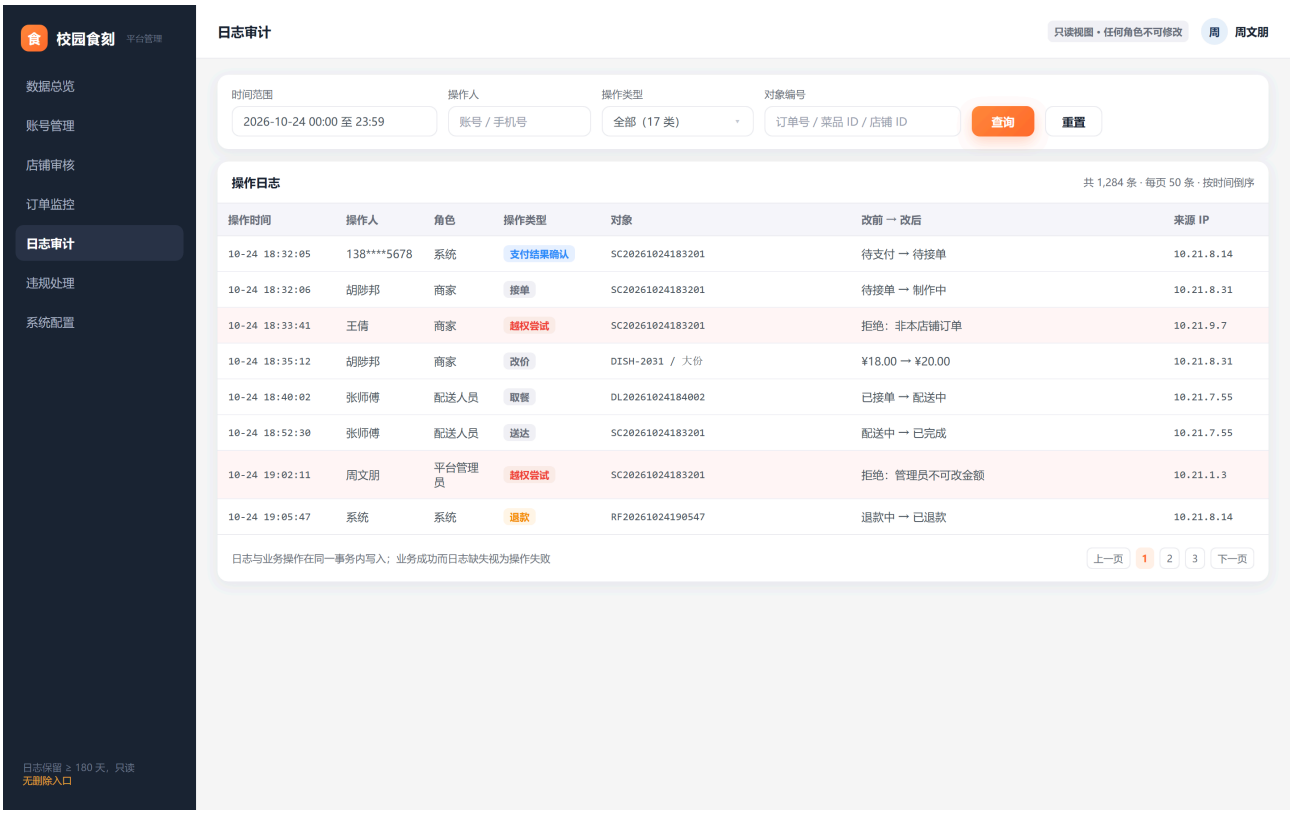


图 27　平台-02 操作日志审计

图 27 支持时间、操作人、操作类型（17 类）、对象编号四条件查询；两行"越权尝试"用浅红底高亮（商家查他人订单被拒、管理员改金额被拒），对应 REQ-002 与 REQ-019"拒绝也要记一条日志"。右上角常驻"只读视图 · 任何角色不可修改"，侧边栏底部写"保留 ≥ 180 天，只读，无删除入口"。

4.6 界面与需求对照

图号	界面	模块	功能要点与对应需求
图 12	店铺与菜品筛选	M2、M3	营业状态、起送配送费、四条件筛选、售罄置灰、最低价（003、004、005）
图 13	规格选择	M3、M4	规格与辣度、规格售罄禁用、价格随规格变化、数量上限（004、006）
图 14	购物车	M4	改价提示、售罄不可选、同店约束、数量与删除（006）
图 15	确认订单	M4、M8	范围校验、配送 / 自取、价格明细、沙箱标识（007、018、020、024）
图 16	支付结果	M5、M6	支付结果、待接单倒计时、超时退款说明、回调为准（007、008、024）
图 17	订单列表	M4、M9	状态分页签、状态化操作、90 天范围（012）
图 18	订单详情	M4、M7	六段进度、骑手脱敏、地址可见性、快照说明（010、012、017）
图 19	收货地址	M8	默认唯一、10 条上限、超范围提示与自取入口（020）
图 20	接单工作台	M2、M6、M7	营业状态切换、倒计时、接单 / 出餐 / 指派（003、008、009）
图 21	拒单原因	M6	四类原因必选、未选不可提交、自动退款（008、011）
图 22	菜品与规格	M3	规格价格与库存、上下架、有订单引用只可下架（004）
图 23	经营统计	M9	净营业额、退款、取消率、销量排行、时效与口径（015）

图号	界面	模块	功能要点与对应需求
图 24	抢单池	M7、M10	在岗、收入、取送点、被抢走状态、到店提醒（009、010）
图 25	配送执行	M7	取餐码、地址可见性、取餐 / 送达 / 失败、改自取（010、017）
图 26	平台数据总览	M12、M9	全平台指标、异常告警、店铺下架（002、015、019）
图 27	日志审计	M11	四条件查询、17 类操作、越权高亮、只读（019、023）

表 9　16 屏界面与模块、需求的对照

4.7 异常态与可用性自检

异常 / 边界	界面表现	需求
菜品售罄	整行置灰、复选框与加减号禁用、标注"不可选中"（图 12、14）	004、006
加购后改价	橙色提示条 + 删除线原价，确认后才可结算（图 14）	006
超出配送范围	红色提示写明距离与半径，并给"到店自取"路径（图 19、15）	020
商家未接单超时	倒计时 + "5 分钟未接单自动取消并全额退款"（图 16、20）	008
骑手抢单失败	按钮变灰为"该单已被接走"（图 24）	009
配送失败	危险色按钮 + 必选原因 + 退款说明（图 25）	010、011
地址不可见	"仅配送期间可见""送达 1 小时后不可见"（图 18、25）	017
越权操作	日志高亮"越权尝试"，界面不提供改金额入口（图 27）	002、019
退款失败	详情显示"退款处理中"，后台进人工队列（图 18、26）	011
统计延迟	固定显示数据截止时间，避免误判漏单（图 23）	015

表 10　异常态与边界情况的界面自检

另按 REQ-018 核对主流程：店铺页（图 12）→ 购物车（图 14）→ 确认订单（图 15）→ 支付结果（图 16），恰好 4 个页面；必填项就地提示、不跳全局错误页；按钮高度均不小于 44 逻辑像素。三点与验收方法一一对应，可直接用于后续可用性测试打分。

五、AI 辅助设计的过程记录

5.1 使用方式与记录

全程使用对话式大模型（deepseek-flash）辅助，共 6 次有实质产出的使用（D01～D06）。每次记录四件事：场景与提示、它给了什么、我判断哪里不对、改成了什么，并附优缺点。之所以把"我判断哪里不对"单列，是因为这部分才是人工价值的所在——只记录"AI 生成了什么"，就成了工具说明书。渲染与校验工具（PlantUML、无头 Chromium、PyMuPDF）不参与判断，只负责把文本变成图、把图变成 PDF 并回读检查。

编号	场景	产出	人工修改的主要动作
D01	架构分层与模块划分	9 模块初稿 + 依赖方向	拆出支付 / 退款、补地址与通知模块、补公共能力，扩为 12 模块
D02	类图	18 类的类图源码	拆为两张、补 4 个实体、修正组合 / 关联、压缩成员与图宽
D03	用例图与活动图	26 个用例 + 3 泳道活动图	收敛到 20 个用例、修正 include / extend、合并校验动作、补泳道
D04	状态图	按 13 条转换生成的图	发现"待配送"死状态，补 2 条转换并回改实验一
D05	UI 页面与控件	14 页清单 + 控件建议	补 2 页、删除评分与优惠券（违反已拒绝需求）、统一规范
D06	一致性与数据核对	7 处不一致	修正图号、金额、编号，并用脚本复核

表 11　6 次 AI 使用记录汇总

5.2 D01：架构与模块划分

场景与提示。把 25 条需求压成一页摘要，要求给出分层架构、模块职责、依赖方向，并指出哪些模块必须同事务。

它给了什么、哪里不对。给出四层架构与 9 个模块，建议订单模块同时承担支付与退款（理由是同属一个聚合根）。三处判断：支付要依赖外部渠道且必须可替换（REQ-024），应独立并带适配层；漏掉"地址与配送范围"（REQ-020 有 10 条上限、默认唯一、距离计算三组规则）；没有说明"扣库存与建单必须同事务"，而这是 REQ-007 的核心。

改成了什么、优缺点。拆成 12 个模块（表 2），把"同事务"提升为架构约束写进依赖规则。优点：分层规范、术语准确、提示了聚合根边界；缺点：倾向教科书划分（支付必须跟订单在一起），对需求编号与边界不敏感，给得出"通用电商模块表"，给不出"这个项目的模块表"。

5.3 D02：类图初稿与重画

场景与提示。列出关键业务对象与三条硬约束（价格快照、流水只追加、配送单唯一持有者），要求输出含属性、方法与关系重数的 PlantUML。

它给了什么、哪里不对。18 个类，关系基本正确。问题有四：图宽 1900px，按 A4 正文宽度打印后字号不足 4pt；缺配送单、退款单、状态流水——它把"配送""退款"当成订单状态而非实体；订单与支付单画成组合，但支付单在订单取消后仍要参与退款；没有表达"明细→规格"是快照引用。

改成了什么、优缺点。拆成两张图，补 4 个实体，改组合为关联、改关联为依赖，压缩成员并启用正交连线，两图缩到 1687 与 1496px。优点：字段齐全（幂等键、渠道交易号这类细节容易漏）；缺点：

把"状态"当"对象"、对打印可读性完全没有概念。这两点后来成了我审查所有 UML 图的固定检查项。

5.4 D03：用例图与活动图

场景与提示。要求用例覆盖 25 条需求、含两个系统参与者；活动图要求五泳道并含库存不足、验签失败、超时未接单三条异常分支。

它给了什么、哪里不对。用例 26 个且把"输入手机号""点击加购"这类界面动作写成用例；把"抢单"画成"指派配送"的 include（应为 extend）；活动图只有 3 个泳道、把三项校验画成三个并列判定，丢掉"共用一次事务"这一关键信息；图宽 3333px。

改成了什么、优缺点。用例收敛到 20 个、修正 include / extend；活动图把三项校验合并为一个动作、补"定时任务"泳道、异常分支全部收在 stop 节点、图宽压到 1252px。优点：生成快、用例命名规范、异常分支想到了一部分；缺点：分不清界面动作与业务用例，对 include / extend 语义掌握不牢，不理解"图要打印在 A4 上"。

5.5 D04：状态图与需求缺口

场景与提示。第一步按 REQ-021 的 13 条转换生成状态图；第二步追加一句："检查这 9 个状态里，是否存在只能进入、无法离开、且不是终态的状态。"

它给了什么、哪里不对。第一步完全忠实于表格，很规整。第二步真的做了出入度分析，指出"待配送"只有一条出口、"已取消"只有退款回执一条出口。前者是真缺口（REQ-011 说制作中及之后可申请取消，含待配送，但转换表没有）；后者是有意设计——订单终结与资金终结分开，由退款单标记失败。

改成了什么、优缺点。补两条转换（表 7 第 11、12 条），回改 REQ-021 并把"退款失败不是缺口"写进图注。优点：换成可判定的提问后，它真能发现缺陷并区分"缺口"与"设计"；缺点：不追问就只是把表格转成图，缺陷会一路带到实现。结论很实用——**它不会怀疑输入，但会认真执行一个明确的检查指令**，人的价值在于提出该检查什么。

5.6 D05：UI 页面与控件

场景与提示。给出模块与角色，要求列出每个角色的关键页面、首屏信息、主控件与异常态呈现方式，移动端 390px、管理端 1440px。

它给了什么、哪里不对。14 页清单，控件建议里包含"菜品卡显示评分与评价数""店铺页放轮播 Banner""支付页展示优惠券入口"。最严重的一条是评分——评价功能在实验一已被甲方明确拒绝（REQ-013），若照抄，报告里就会出现一个需求中不存在的模块；同时清单漏了"支付结果"与"拒单原因"两页，而这两页恰好承载 REQ-007、REQ-008 的验收点；管理端只给 1 页，日志审计没有归属。

改成了什么、优缺点。定为 16 屏，删除评分、评价、优惠券、Banner，补支付结果、拒单原因、日志审计；改为自建 HTML/CSS 组件库 + 2 倍导出（理由见 4.1）。优点：页面清单的"首屏 / 控件 / 异常态"结构成了每屏说明的模板，它列的异常态也在需求范围内；缺点：对"本项目明确不做"毫无记忆，视觉建议偏营销页，与校园小店"快、准、别出错"的诉求不一致。

5.7 D06：一致性与数据核对

场景与提示。报告涉及 27 个图号、14 个表号、25 个需求编号与跨屏金额，要求它只找不一致、不要改写：图号表号是否连续且引用一致，金额是否自洽，状态名是否与 REQ-021 一致。

它给了什么、哪里不对。列出 7 处，5 处为真：图号误写、合计金额与说明不一致、"待配送"被写成"待取餐"、表号重复、某屏描述出现"评价"字样；另 2 处是它在没有全文时的误报。

改成了什么、优缺点。5 处全部修正，并另写脚本抽取"图 N / 表 N / REQ 号"查重与连续性——机器查结构、人查语义。优点：对编号与数字极其可靠，不会看多遍后"自动脑补正确"；缺点：会误报，也会漏掉语义矛盾，只能当第一道筛子。

5.8 大模型的能力边界

擅长的：给结构骨架（分层、模块候选、页面清单），让人从空白页变成"有东西可改"；术语与格式规范化（口语需求改写成"对象 + 条件 + 判断标准"，设计写成合法 PlantUML）；穷举候选方案；按可判定清单做检查；文字润色与术语统一。

不擅长的：判断需求真伪（把"行业里通常有"当成"本项目该有"，评分与优惠券就是这么来的）；发现输入自身的矛盾（除非被明确要求做可达性/一致性分析）；工程可行性（打印字号、图宽、事务边界、并发锁）；业务常识（日均 150 单、老板自己配送）；记住"明确不做"的清单。

由此得到一条操作原则：**把"生成"交给它，把"判定"留给人，把"检查"尽量变成机器可执行的规则。**

5.9 让人只做检查校对的工作流

结论是：**部分可以，但"检查"必须从逐页看升级为可判定的核验**，否则人既累又不可靠。据此整理五阶段流水线：**S0 输入规格化**（需求有编号、验收标准与黑名单，人主导，1～2 小时）→ **S1 候选生成**（每类产物出 1～3 个候选，AI 主导）→ **S2 机器校验**（脚本重跑：编译、图宽算字号、覆盖矩阵、编号连续性、金额自洽、状态可达性）→ **S3 人工闸门**（只回答四类判断：需求是真的吗、取舍代价可接受吗、验收标准合理吗、界面符合场景吗）→ **S4 归档回归**（每次改动进 Git 并标注 AI / 人工，下轮只对差异重跑）。本次实验中 S2 做了 4 项、S3 的判断集中在 D01 / D04 / D05 三次、S4 体现为 6 次提交。

#	检查项	可自动	说明
1	PlantUML 能否编译、语法合法	是	渲染失败即报错，可批量执行
2	图宽 ÷ 字号换算的打印字号 ≥ 6pt	是	本次据此重画了类图与活动图
3	状态机是否存在不可达终态	是	遍历出入度即可，D04 由此发现
4	每条需求是否被模块与图覆盖	是	用覆盖矩阵做集合差
5	图号、表号、需求编号连续且引用一致	是	D06 的脚本化版本
6	跨界面金额、状态、时间线自洽	是	把关键数字抽成表比对
7	正文功能是否落在"明确不做"黑名单	是	关键词比对，可拦住评分 / 优惠券
8	界面是否符合设计规范（色板、字号、触摸目标）	部分	颜色尺寸可查，层级是否清晰不能
9	需求是否真实存在、是否甲方确认	否	必须回到访谈与甲方确认
10	模块边界与架构取舍是否合理	否	取决于团队规模、运维能力与周期
11	验收标准数值是否合理（95 分位 800ms）	否	需业务与成本共同判断
12	界面是否适合真实使用场景（后厨戴手套点屏）	否	"商家端按钮要大"属此类判断

表 12 检查清单：哪些能交给机器，哪些必须由人判断

结论：在需求清晰、检查项可判定的前提下，人只做 S3 的四个判断加 S2 的抽检是可行的；但三件事无法外包——**确认需求真实存在**（评分功能的教训）、**承担取舍后果**、**定义什么叫"对"**。另有两个风险：误报会养成"橡皮图章"式校对；模型同源会让设计趋同成"标准电商"。

六、实验总结

6.1 收获与不足

收获有四。第一，需求到设计不是翻译而是补全：画状态图才发现"待配送"没有取消出口，需求逐条自洽、合起来却有缺口，设计阶段的价值正在于把需求放进同一个模型里跑一遍。**第二，图的"可读性"是设计质量的一部分**：18 个类、1900px 宽的类图打印后字号不足 4pt，等于没画；拆图、删成员、压标签之后信息没少，可读性完全不同。**第三，模块边界要用"数据 + 事务"来划**：支付看起来天经地义属于订单，但它依赖外部渠道、要独立退款、要可切换，绑在订单里 REQ-024 就无法满足。**第四，横切规则要写进界面**：脱敏、地址可见期限、倒计时、只读日志，写在需求里只有一句话，不体现在界面上就不会被遵守。

不足也有四项。（1）只有逻辑模型，缺建表语句、索引与分页 SQL，REQ-016 的指标最终要靠索引落地；（2）接口契约未定义，403 / 409 语义还没固化成错误码表；（3）权限只到模块级，缺"接口 × 角色 × 数据范围"矩阵；（4）REQ-018 要求 10 名首次使用者至少 8 人独立完成下单，本次只做了四页面与 44px 的设计层面自检，未组织真实测试。以上留到实验三。

6.2 与实验一的回改

本次只对实验一做了一处实质性回改：REQ-021 的转换表补了两条。"待配送"原本只能经"分配成功"离开，而 REQ-011 规定制作中及之后顾客可申请取消、商家同意后取消，两者合起来意味着"出餐后无人接单的订单永远卡住"。这不是实现细节而是需求缺口，因此在需求文档中补记，并按课程要求用 Git 提交（C3）。

位置	修改前	修改后	原因
REQ-021 转换表	13 条转换，"待配送"只有第 10 条一条出口	追加第 14 条（商家取消 • 无人可派）与第 15 条（顾客申请经商家同意），均为"待配送 → 已取消"	消除死状态，与 REQ-011 对齐；状态图、表 7、活动图同步更新
REQ-021 文末	无	追加"设计阶段补记"：说明两条转换的来源，并澄清"退款失败停在已取消"不是缺口	避免实现者误以为要新增订单状态
验收方法	"对表中 13 条转换逐条执行"	"对表中 15 条转换逐条执行"	与转换表保持一致

表 13 与实验一的回改记录（对应 Git 提交 C3）

其余内容未改：REQ-013、REQ-014 仍为"已拒绝"，REQ-024 仍"待确认"（沙箱通道 + 界面显著标记），范围与性能指标照旧。设计阶段产生的"理解性澄清"（如"制作中及之后"含待配送、退款失败不是状态缺口）没有写进需求文本，只写进图注与本报告，避免把解释混进需求。

七、版本与提交

7.1 仓库与目录

沿用实验一的仓库，分支 `main`，地址 `https://github.com/yui060814zwp-creator/SE-lab`。实验一内容（`chat.txt`、`requirements.md`）保持原位，实验二产物在 `lab2/`；唯一改动实验一文件的是 REQ-021（6.2）。`lab2/` 下为 `实验报告.pdf`、`UML/`（11 张 puml + 11 张 png）、`UI界面/`（16 屏）。

7.2 提交记录

为了让"AI 生成"与"人工修改"在版本史上可分辨，没有一次性提交：C0 归档实验一报告，C1～C5 是实验二的五个阶段。每个提交的 diff 就是一次真实修改。

提交	提交号	内容	性质	说明
C0	`c239534`	report.pdf（实验一报告归档）	归档	此前未入库，本次补齐
C1	`ec7e1d0`	UML 的 AI 初稿（10 张图，类图为一整张）	AI 生成	D01～D03 的首次输出，未修改直接入库以便对比
C2	`7ad65e6`	修订后 UML 全套 11 图（类图拆两张，重画活动图 / 状态图 / 组件图 / 用例图）	人工修改	修正组合 / 关联、补 4 个实体、压缩图宽
C3	`3f212ca`	requirements.md：REQ-021 追加第 14、15 条与设计阶段补记	人工判断	D04 发现死状态后的需求回改
C4	`c057140`	16 屏界面 + 界面源码	AI 初稿 + 人工修改	删评分 / 优惠券、补 2 屏、统一色板与间距
C5	本报告所在提交	实验报告 PDF、报告源码与过程材料	AI 辅助 + 人工核对	D06 的核对在提交前完成：`lab2-final` 指向该提交

表 14 Git 提交记录（C1 打 `lab2-draft`，C5 打 `lab2-final`）

7.3 AI 生成与人工修改的对应

分布是这样的：**结构性内容以 AI 为主**（模块候选、类图源码、PlantUML 语法、页面清单、文字润色），**判断性内容全部由人完成**（支付是否独立、状态机死状态、是否允许评分入口、图的打印可读性、界面在真实场景下的可用性）。C1 到 C2 的 diff 是最直观的证据：10 张图变 11 张、类图一分为二、连线语义被改正。提交信息统一以 `lab2:` 开头并写明性质，仅看 `git log` 也能分辨哪一步是生成、哪一步是判断。

周文朋　241220167　2026 年 11 月 12 日